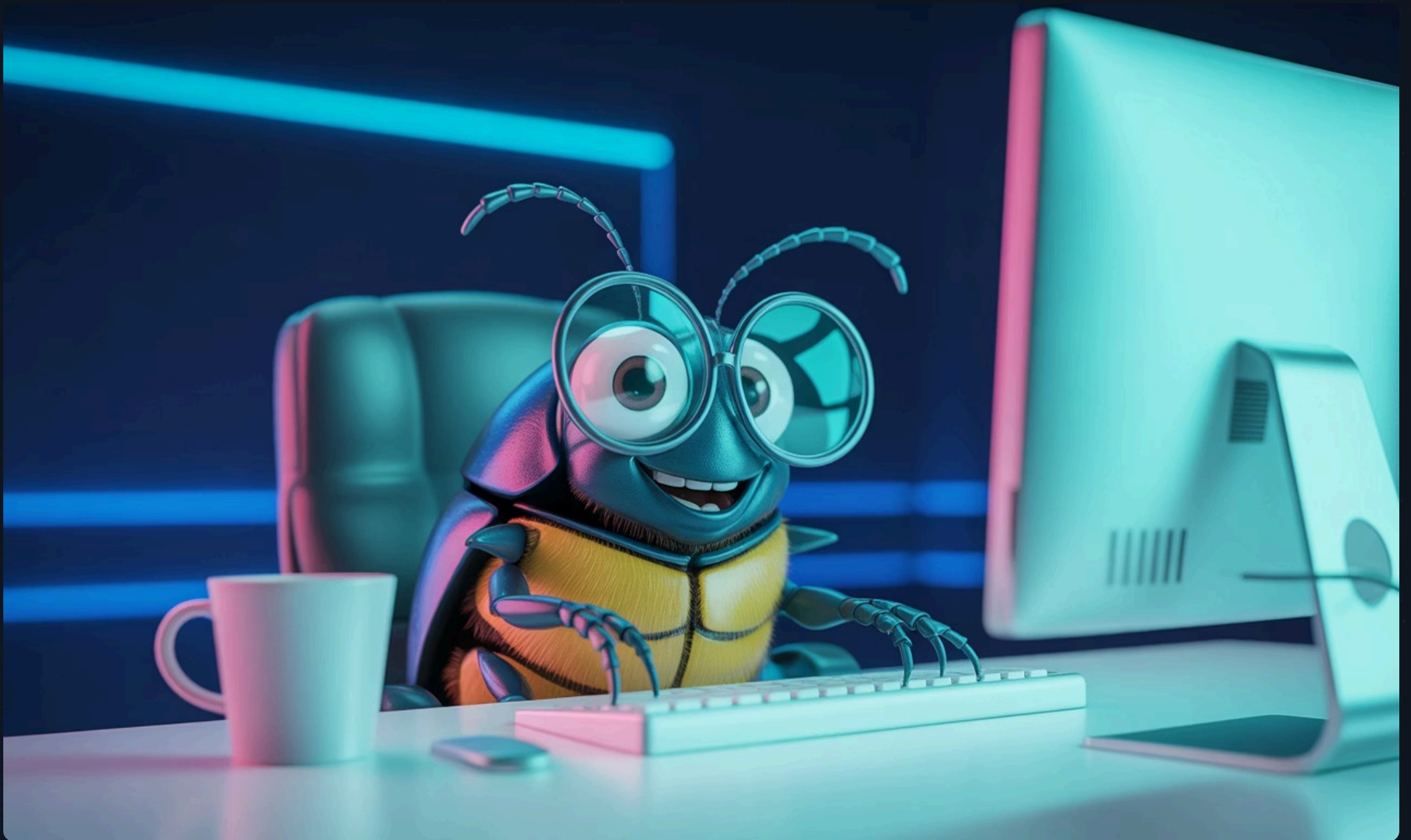


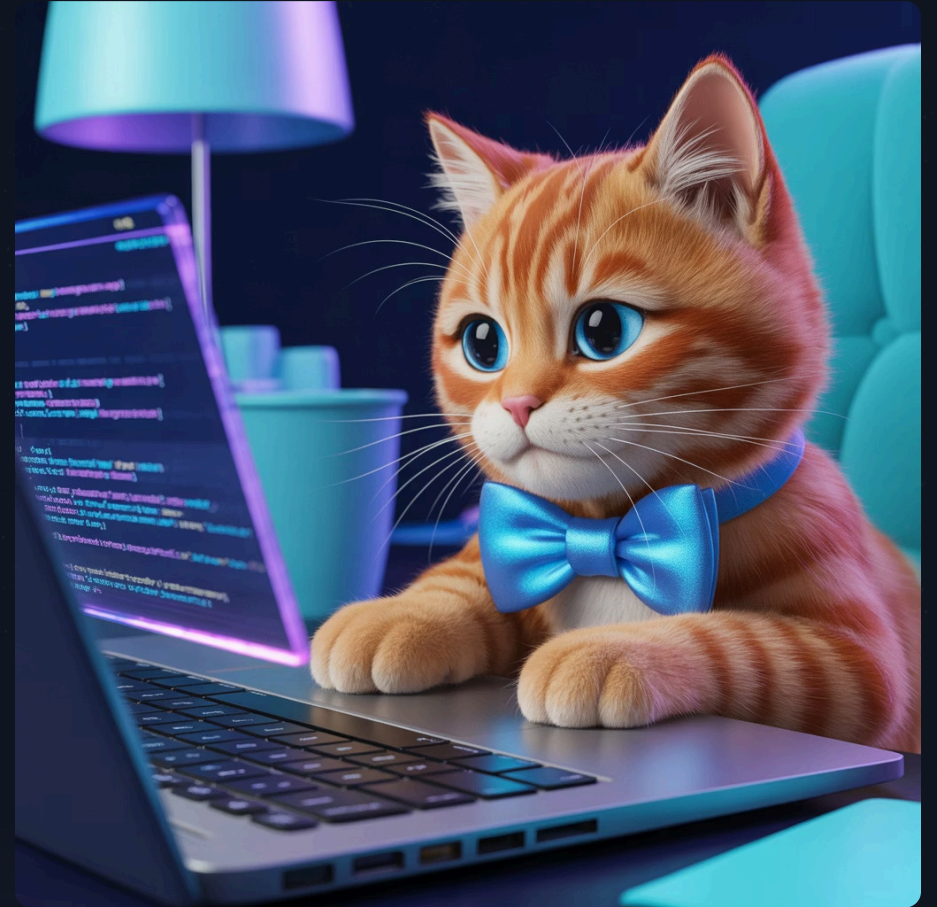
Practice Exercise: Adding a Subtraction Method



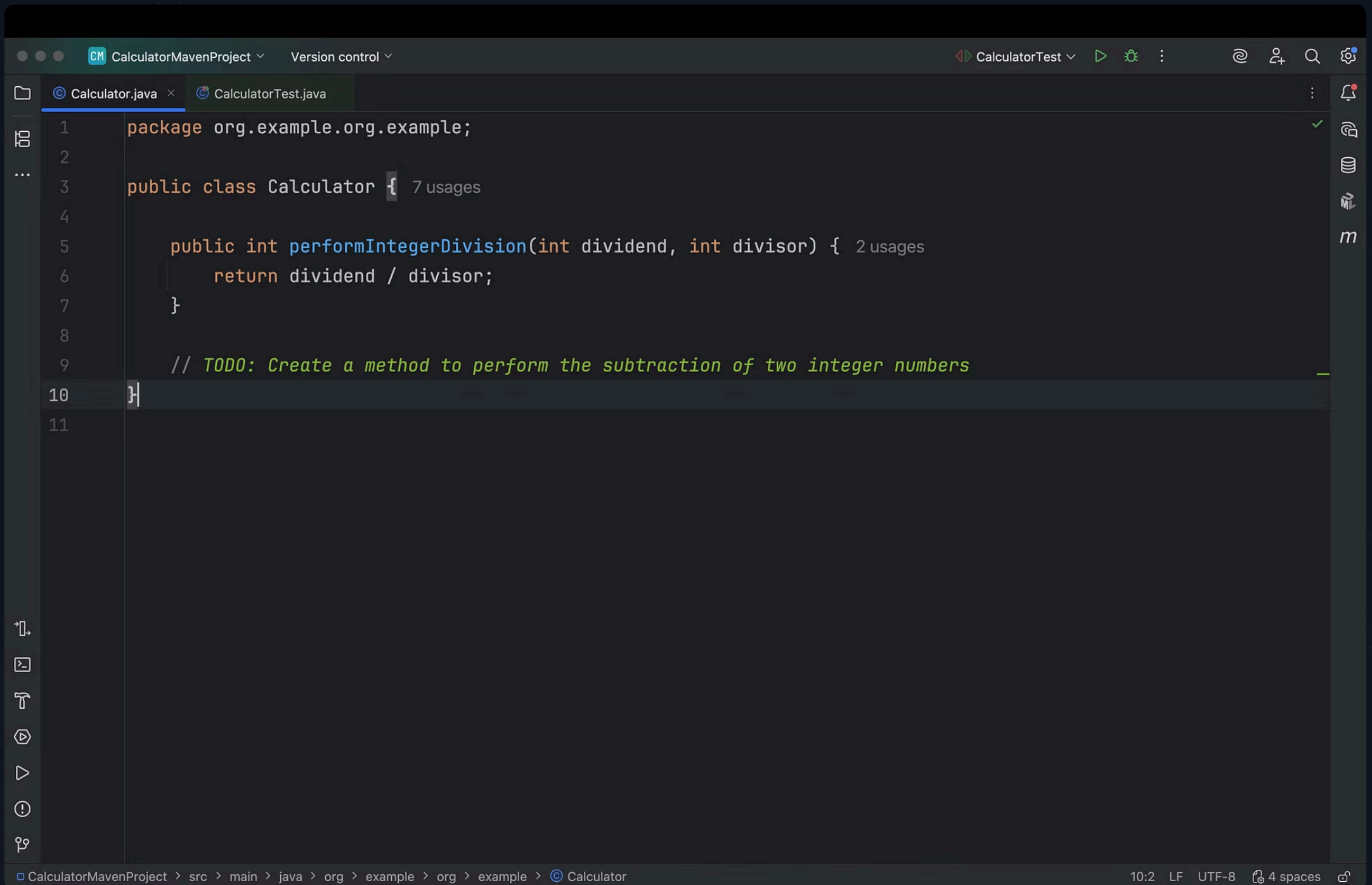
You will now extend the Calculator class with a subtraction operation. This reinforces how to add new behavior and test it independently. The goal is to implement a simple method that returns the result of $a - b$. After implementing, you should test it using JUnit in the same project.

Implementing the Subtraction Method

- Open the Calculator class and create a new `performIntegerSubtraction` **method**
- The method should accept two integers: **minuend** and **subtrahend**
- The body of the method simply returns `minuend - subtrahend`
- Keep the logic simple, without additional validation for now



Implementing the Subtraction Method



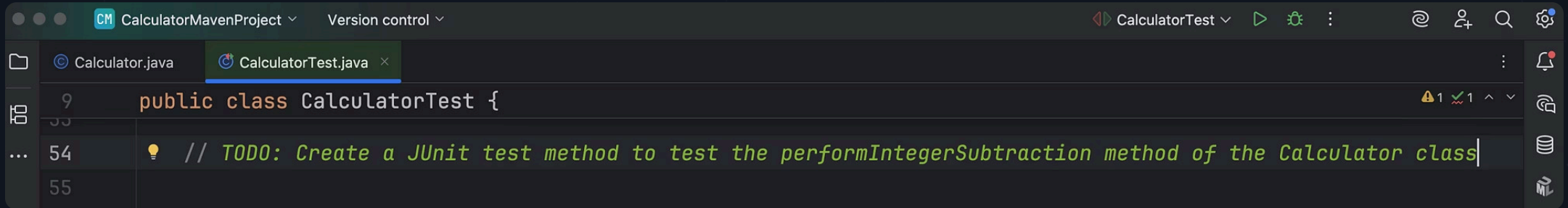
The screenshot shows an IDE window for a project named "CalculatorMavenProject". The "Calculator.java" file is open, displaying the following code:

```
1 package org.example.org.example;
2
3 public class Calculator { 7 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 2 usages
6         return dividend / divisor;
7     }
8
9     // TODO: Create a method to perform the subtraction of two integer numbers
10
11 }
```

The IDE interface includes a sidebar on the left with icons for Explorer, Search, Run and Debug, and other tools. The top bar shows the project name, version control status, and a "CalculatorTest" tab. The bottom status bar indicates the file path: "CalculatorMavenProject > src > main > java > org > example > org > example > Calculator", the cursor position "10:2", and the encoding "UTF-8".

Open the Calculator class and add a new test method

Writing the JUnit Test



```
CalculatorMavenProject Version control CalculatorTest  
Calculator.java CalculatorTest.java  
9 public class CalculatorTest {  
54 // TODO: Create a JUnit test method to test the performIntegerSubtraction method of the Calculator class  
55
```



Open the CalculatorTest class and add a new test method



Use **Arrange–Act–Assert** to keep the test easy to read and maintain



Arrange

prepare input values and a Calculator instance



Act

call the new method with the prepared values



Assert

verify that the returned result matches the expected subtraction

Conclusion



Practice Makes Perfect

This exercise helps you practice creating methods and testing them independently

Clear Structure

Using Arrange–Act–Assert ensures clarity and consistency in test structure

Verification

Running the test confirms both your implementation and understanding of unit testing

Open Floor Discussion



Edge Cases

How would you test edge cases, such as subtracting negative numbers?

Boundary Values

Should subtraction tests include very large or very small integers?

Code Reuse

How might you reuse setup code across multiple arithmetic tests?